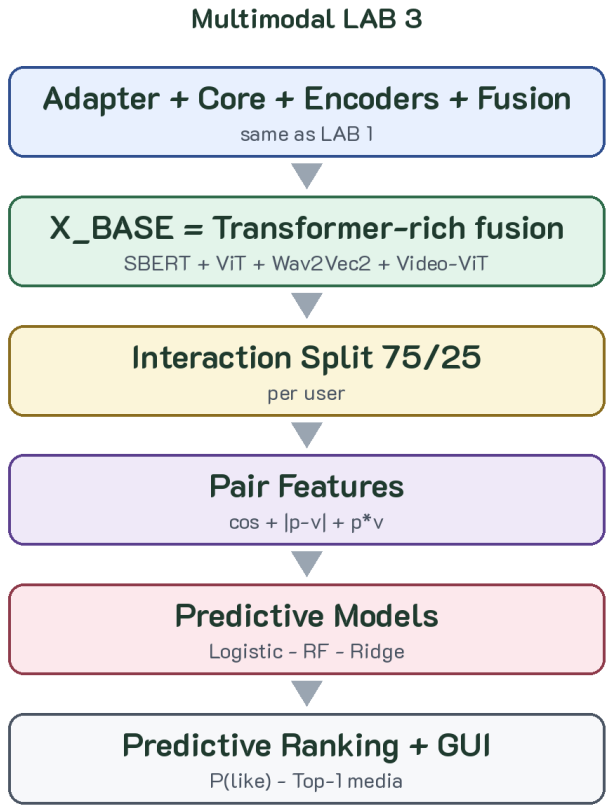
LAB 6:
Multimodal Predictive

จากหลายสื่อสู่โมเดลทำนาย

204729
Recommender Systems

ผู้ช่วยศาสตราจารย์ ดร.ดุษฎี ประเสริฐธิติพงษ์

End-to-End Multimodal Predictive Pipeline



อธิบายแนวคิดของ Source Code ทีละ Part

Adapter - Vector - Fusion - Profile - Pair Features - Model - Ranking - Evaluation

COMPUTER SCIENCE
Chiang Mai University

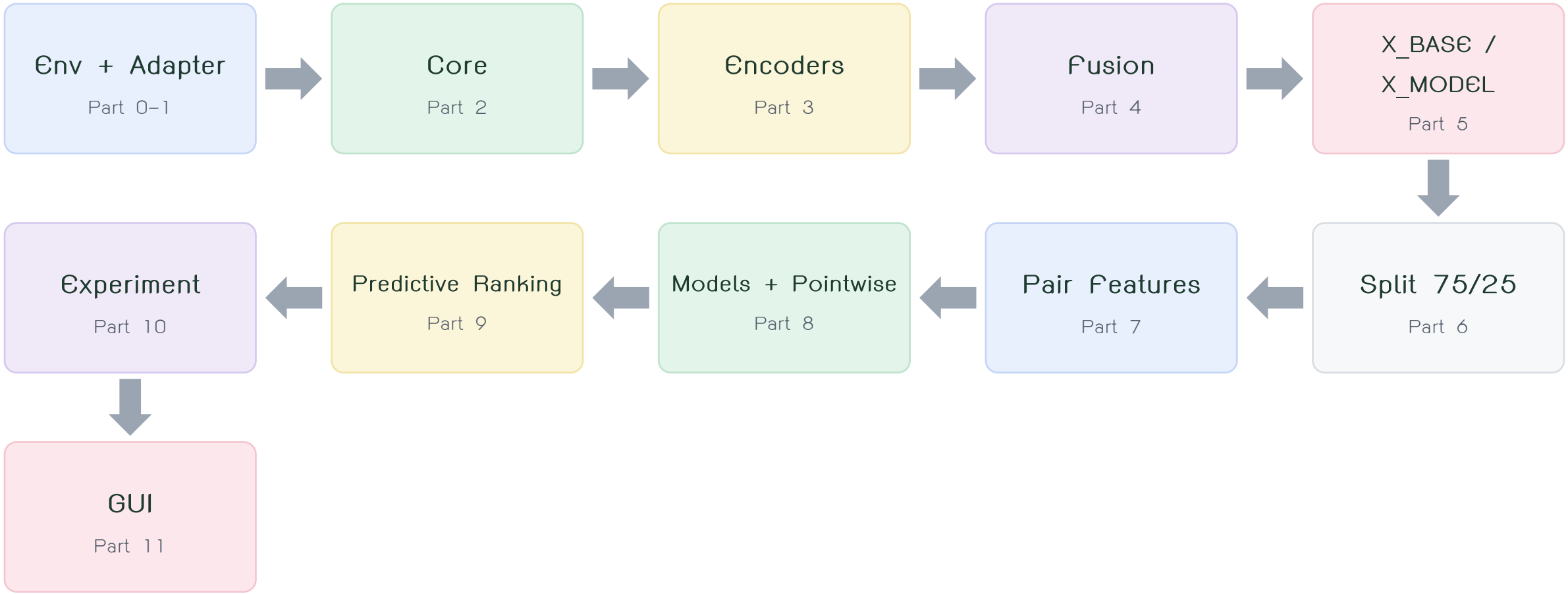
Multimodal LAB 3 ต่อยอดจาก LAB 1-2 อย่างไร

ภาพรวม

- Part 0-4 เหมือน LAB 1-2 (Adapter, Core, 9 Encoders, Fusion) ; LAB 3 ใช้ `make_profile`, `transform_vectors`, `early_fusion` และ metric เดิม แต่ **ไม่ใช้** `evaluate_vector_space` เพราะเปลี่ยนวิธีจัดอันดับ
- Pipeline: Dataset Adapter ⟶ Multimodal Vector ⟶ Transformation/Fusion ⟶ User Profile ⟶ Pair Features ⟶ Predictive Model ⟶ Ranking ⟶ Evaluation
- Part 5: `X_BASE` = early fusion แบบ Transformer-rich (SBERT + ViT + Wav2Vec2 + Video-ViT = 2,688 มิติ) ⟶ `X_MODEL = L2`
- Part 6: split ระดับ interaction 75/25 ต่อผู้ใช้ ⟶ Part 7 pair features `[cos, |p - v|, p ⊙ v]` = 1 + 2,688 + 2,688 = 5,377 มิติ จากโปรไฟล์ที่สร้างจาก train เท่านั้น
- Part 8: Classification (like / not-like) + Regression (rating ⟶ MAE/RMSE) ; Part 9: ใช้ P(like) จัดอันดับแล้ววัดด้วย metric บทที่ 2 ; Part 10: Profile × Transformation × Model ⟶ AUC ; Part 11: GUI แสดง P(like) + สื่อของอันดับ 1

โครงเดียวกับ Text LAB 3 แต่ item vector เปลี่ยนจาก SBERT 384 มิติ เป็น fused multimodal 2,688 มิติ

โครงสร้าง Notebook Multimodal LAB 3



Part 5-11 คือส่วนใหม่ ; Part 0-4 คือโค้ดเดิม

แผนที่ของ Source Code Multimodal LAB 3

Part	หน้าที่	ฟังก์ชัน / ตัวแปรสำคัญ	สิ่งใหม่
0–4	Adapter, Core, Encoders, Fusion	`load_dataset`, `make_profile`, `transform_vectors`, `VECTOR_LIBRARY`, `early_fusion`	เหมือน LAB 1–2
5	เลือก multimodal representation	`X_BASE` (Transformer-rich), `X_MODEL = transform_vectors(X_BASE, "L2")`	ใหม่
6	Interaction train/test split	`train_r`, `test_r` (75/25 ต่อผู้ใช้, seed 42)	ใหม่
7	Pair features	`PROFILE_FOR_MODEL`, `pair_feature`, `make_pairs`, `Ftr/Fte`, `yctr/ycte`, `yrtr/yrte`	ใหม่ — รับ `profile_method` เป็นพารามิเตอร์
8	โมเดล + pointwise metric	`CLF`, `REG`, `cr`, `rr`	Accuracy/Precision/Recall/F1/AUC, MAE/RMSE
9	Predictive ranking + metric บทที่ 2	`best`, `predictive_rank`, `rank_detail`, `summary`	score = P(like)
10	Experiment	`exp` (Trans × Profile × Model ⟶ AUC)	`clone` โมเดล
11	GUI	`gui_pred`	P(like) + Top-1 image/audio/video

1

Representation และ Train/Test Split

Part 5 – Part 6: fused vector สำหรับโมเดล แล้วแบ่ง rating 75/25 ต่อผู้ใช้

- `X_BASE` ต่างจาก `X_fused` ของ LAB 1: ใช้ ViT แทน ResNet สำหรับภาพ
- `train_r` เป็นแหล่งเดียวของโปรไฟล์และ feature ทั้งหมด

Part 5: X_BASE และ X_MODEL

```
# Transformer-rich / hybrid representation
X_BASE = early_fusion({
    "text":  X_text_sbert,          # 384
    "image": X_image_vit,           # 768
    "audio": X_audio_transformer,   # 768
    "video": X_video_vit            # 768
})
X_MODEL = transform_vectors(X_BASE, "L2")
print(X_MODEL.shape)                # (12, 2688)
```

อธิบายโค้ด

เลือก representation เดียวสำหรับ pipeline

- `X_BASE` = Transformer-Multimodal ของ LAB 2 (SBERT + ViT + Wav2Vec2 + Video-ViT) ผ่าน `early_fusion` (√w = 1) ⟶ 2,688 มิติ
- `X_MODEL` = L2 (early_fusion L2 ให้แล้ว จึงเท่าเดิม) ; `X_BASE` เก็บไว้ให้ Part 10 ทดลอง `Center+L2`
- ทำไมไม่ใช้ CNN family: TextCNN อ่อนและ ResNet ให้มิติต่างกัน — เล่มนี้เน้น pipeline จึงเลือกชุดที่คุณภาพเวกเตอร์น่าเชื่อถือที่สุด
- มิติสูง (2,688) ⟶ pair features จะมี 5,377 มิติ กับข้อมูลไม่กี่สิบแถว ⟶ โมเดลต้อง regularize และตีความ AUC อย่างระวัง

Part 6: Interaction Train/Test Split 75/25 ต่อผู้ใช้

```
rng = np.random.default_rng(42)
tr = []; te = []
for uid, g in ratings.groupby("user_id"):
    ids = g.index.to_numpy()
    if len(ids) < 2: tr += list(ids); continue        # rating เดียว ->
            train ทั้งหมด
    n = max(1, int(round(len(ids) * 0.25)))
    held = set(rng.choice(ids, size=n, replace=False))
    te += list(held); tr += [i for i in ids if i not in held]
train_r = ratings.loc[tr].copy(); test_r = ratings.loc[te].copy()
print("train", len(train_r), "test", len(test_r))
```

อธิบายโค้ด

ซ่อน 25% ของ rating ต่อคน (ไม่เฉพาะ positive)

- ทุกผู้ใช้อยู่ทั้งใน train และ test ⟶ test มีทั้ง rating สูงและต่ำ = label 0/1 และค่า rating จริง สำหรับ classifier และ regressor
- ต่างจาก LOO ของ LAB 1–2 (ซ่อน positive 1 เรื่อง): ที่นี่ relevant ต่อคนใน test อาจมีหลายเรื่องหรือไม่มีเลย
- `rng` seed 42 ⟶ ทำซ้ำได้ ; ผู้ใช้ที่มี rating เดียวไม่ถูกทดสอบ
- **หลักการ:** โปรไฟล์และ pair features ทุกตัวสร้างจาก `train_r` เท่านั้น ⟶ ไม่มี leakage จาก test

2

User–Item Pair Features

Part 7: จากคู่ (user, item) เป็นเวกเตอร์ feature 5,377 มิติ

- `pair_feature(uid, iid, X, rdf, profile_method)` → `[cos, |p - v|, p ⊙ v]`
- `make_pairs(df, X, history, profile_method)` → `(F, y_cls, y_reg, meta)`

Part 7: pair_feature และ make_pairs

```
PROFILE_FOR_MODEL = "Rating-centered"

def pair_feature(uid, iid, X, rdf, profile_method):
    p = make_profile(uid, X, rdf, profile_method)              #
            โปรไฟล์จาก history ที่ส่งเข้ามา
    v = X[ITEM_INDEX[str(iid)]]
    cos = float(np.dot(p, v) / (max(np.linalg.norm(p) *
            np.linalg.norm(v), 1e-12)))
    return np.concatenate([[cos], np.abs(p - v), p * v])       # 1 +
            2688 + 2688

def make_pairs(df, X, history, profile_method):
    F = []; yc = []; yr = []; meta = []
    for _, r in df.iterrows():
        iid = str(r["item_id"]); uid = str(r["user_id"])
        if iid not in ITEM_INDEX: continue
        F.append(pair_feature(uid, iid, X, history, profile_method))
        yr.append(float(r["rating"])); yc.append(int(float(r["rating"])
                >= 4)); meta.append((uid, iid))
    return np.asarray(F), np.asarray(yc), np.asarray(yr), meta

Ftr, yctr, yrtr, _       = make_pairs(train_r, X_MODEL, train_r,
        PROFILE_FOR_MODEL)
Fte, ycte, yrte, meta_te = make_pairs(test_r,  X_MODEL, train_r,
        PROFILE_FOR_MODEL)
print(Ftr.shape, Fte.shape)
```

อธิบายโค้ด

feature ที่ให้โมเดลถ่วงน้ำหนักต่อมิติ

- cos = คะแนน content-based เดิม ; |p − v| และ p ⊙ v ต่อมิติ ⟶ โมเดลเรียนรู้ว่ามิติใดของ fused space (ส่วน text? image? audio?) สำคัญต่อ “ชอบ” — เทียบกับ cosine ที่ให้น้ำหนักทุกมิติเท่ากัน
- เพราะ `X_MODEL` เป็น concat ของ 4 สื่อ มิติ 0–383 = text, 384–1151 = image, ... ⟶ น้ำหนักของ Logistic ต่อช่วงมิติบอก **modality importance** ได้ (ต่อยอดได้)
- ต่างจาก Text LAB 3: `pair_feature` รับ `profile_method` เป็น **พารามิเตอร์** (ไม่อ่าน global) ⟶ Part 10 สลับ profile ได้สะอาดกว่า
- กัน leakage: `make_pairs(test_r, X, train_r, ...)` — โปรไฟล์ของแถว test มาจาก train_r ; แถว train ใช้ train_r เอง (optimistic เล็กน้อย)
- target: `yc = rating ≥ 4`, `yr = rating` ; `meta` เก็บ (uid, iid)

โครงสร้าง feature ของคู่ (user, item) ใน fused space

ส่วน	สูตร	มิติ	ช่วงมิติที่มาจากสื่อ
cos	$p \cdot v \;/\; (\|p\|\|v\|)$	1	รวมทุกสื่อ
$\|p - v\|$	abs(p − v)	2,688	text 0–383 • image 384–1151 • audio 1152–1919 • video 1920–2687
$p \odot v$	p × v	2,688	ช่วงเดียวกัน
รวม	concatenate	5,377	input ของ Logistic / RF / Ridge

feature มากกว่าตัวอย่างหลายสิบเท่า ⟶ เสี่ยง overfit ; Logistic มี L2 regularization ค่าเริ่มต้น (C=1)

3

Predictive Models และ Predictive Ranking

Part 8 – Part 9: pointwise metric แล้วใช้ P(like) จัดอันดับวัดด้วย metric บทที่ 2

- `CLF` = Logistic / RandomForest ; `REG` = Ridge / RandomForest
- `predictive_rank(uid, model, X, history, profile_method)`

Part 8: ฝึกและวัด classifier / regressor (1/2)

```python
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.ensemble import RandomForestClassifier,
        RandomForestRegressor
from sklearn.metrics import accuracy_score, precision_score,
        recall_score, f1_score

CLF = {"LogisticRegression": LogisticRegression(max_iter=2000,
        class_weight="balanced"),
       "RandomForestClassifier":
               RandomForestClassifier(n_estimators=200,
                       random_state=42, class_weight="balanced")}
REG = {"Ridge": Ridge(alpha=1.0),
       "RandomForestRegressor": RandomForestRegressor(n_estimators=200,
               random_state=42)}

cr = []
for name, m in CLF.items():
```

อธิบายโค้ด

สองงานบน test เดียวกัน

- **Classification** (pointwise ต่อคู่): Accuracy / Precision / Recall / F1 ที่ threshold 0.5 และ AUC จาก probability ; `class_weight="balanced"` ชดเชย class imbalance
- **Regression**: MAE / RMSE ของ rating ที่ทำนาย
- โมเดลใน `CLF` ถูก `fit` ที่นี่และเก็บสถานะ ⟶ `CLF["LogisticRegression"]` ถูกใช้ต่อใน Part 9 และ GUI
- ไม่มี scaler: feature จากเวกเตอร์ L2 อยู่ในสเกลใกล้กัน ; `max_iter=2000` ให้ Logistic ลู่เข้ากับ 5,377 มิติ
- AUC ดีในระดับคู่ ≠ Top-K ดีต่อผู้ใช้ ⟶ Part 9

Part 8: ฝึกและวัด classifier / regressor (2/2)

```
    m.fit(Ftr, yctr); pr = m.predict_proba(Fte)[:, 1]; y = (pr >=
            .5).astype(int)
    cr.append({"Model": name, "Accuracy": accuracy_score(ycte, y),
               "Precision": precision_score(ycte, y, zero_division=0),
               "Recall": recall_score(ycte, y, zero_division=0),
               "F1": f1_score(ycte, y, zero_division=0),
               "AUC": roc_auc_score(ycte, pr) if len(np.unique(ycte))
                      == 2 else np.nan})
display(pd.DataFrame(cr).round(4))

rr = []
for name, m in REG.items():
    m.fit(Ftr, yrtr); pr = m.predict(Fte)
    rr.append({"Model": name, "MAE": mean_absolute_error(yrte, pr),
           "RMSE": mean_squared_error(yrte, pr) ** 0.5})
display(pd.DataFrame(rr).round(4))
```

อธิบายโค้ด

สองงานบน test เดียวกัน

- **Classification** (pointwise ต่อคู่): Accuracy / Precision / Recall / F1 ที่ threshold 0.5 และ AUC จาก probability ; `class_weight="balanced"` ชดเชย class imbalance
- **Regression**: MAE / RMSE ของ rating ที่ทำนาย
- โมเดลใน `CLF` ถูก `fit` ที่นี่และเก็บสถานะ ⟶ `CLF["LogisticRegression"]` ถูกใช้ต่อใน Part 9 และ GUI
- ไม่มี scaler: feature จากเวกเตอร์ L2 อยู่ในสเกลใกล้กัน ; `max_iter=2000` ให้ Logistic ลู่เข้ากับ 5,377 มิติ
- AUC ดีในระดับคู่ ≠ Top-K ดีต่อผู้ใช้ ⟶ Part 9

Part 9: predictive_rank และ loop ประเมินต่อผู้ใช้ (1/2)

```
best = CLF["LogisticRegression"]

def predictive_rank(uid, model, X, history, profile_method):
    seen = set(history.loc[history["user_id"].astype(str) == str(uid),
            "item_id"].astype(str))
    cand = [i for i in items["item_id"] if i not in seen]
    if not cand: return [], {}
    F = np.vstack([pair_feature(uid, i, X, history, profile_method) for
            i in cand])
    pr = model.predict_proba(F)[:, 1]
    sc = dict(zip(cand, pr))
    return sorted(cand, key=lambda i: sc[i], reverse=True), sc

rows = []; all_recs = []
for uid, g in test_r.groupby("user_id"):
    rel = set(g.loc[g["rating"] >= 4, "item_id"].astype(str))
    if not rel: continue
    ranked, sc = predictive_rank(uid, best, X_MODEL, train_r,
```

อธิบายโค้ด

P(like) เป็นคะแนนจัดอันดับ

- `predictive_rank`: candidate = ทุกเรื่องที่ไม่อยู่ใน train ของผู้ใช้ (รวมเรื่องใน test) ⟶ pair features ⟶ `predict_proba` ⟶ เรียงตาม P(like)
- relevant = เรื่องใน test ที่ rating ≥ 4 (หลายเรื่องได้) ⟶ Precision@3 / Recall@3 / MAP@3 มีความหมายเต็ม ; ผู้ใช้ที่ test ไม่มี positive ถูกข้าม
- metric ทั้งหมดจาก Core เดิม ; ILD วัดใน `X_MODEL` (fused) ; Calibration เทียบกับ `train_r`
- ตัวเลขนี้เทียบกับ LAB 1-2 ได้เชิงโครงสร้าง แต่ **split ต่างกัน** (75/25 vs LOO) ต้องระบุเสมอ
- `best` เปลี่ยนเป็น RF ได้บรรทัดเดียว

Part 9: predictive_rank และ loop ประเมินต่อผู้ใช้ (2/2)

```
            PROFILE_FOR_MODEL)
    top = ranked[:3]; all_recs += top
    rows.append({"user_id": uid, "Precision@3": precision_at_k(top,
            rel, 3),
                 "Recall@3": recall_at_k(top, rel, 3), "F1@3":
                        f1_at_k(top, rel, 3),
                 "HR@3": hr_at_k(top, rel, 3), "MRR":
                        reciprocal_rank(ranked, rel),
                 "MAP@3": ap_at_k(top, rel, 3), "nDCG@3":
                        ndcg_at_k(top, rel, 3),
                 "ILD": ild(top, X_MODEL), "Novelty": novelty(top),
                 "Calibration": calibration_genre(uid, top, train_r)})
rank_detail = pd.DataFrame(rows)
summary = rank_detail.mean(numeric_only=True).to_dict()
summary["Coverage"] = coverage(all_recs); summary["Gini"] =
        gini(all_recs)
display(rank_detail.round(4));
        display(pd.Series(summary).to_frame("mean").round(4))
```

อธิบายโค้ด

P(like) เป็นคะแนนจัดอันดับ

- `predictive_rank`: candidate = ทุกเรื่องที่ไม่อยู่ใน train ของผู้ใช้ (รวมเรื่องใน test) ⟶ pair features ⟶ `predict_proba` ⟶ เรียงตาม P(like)
- relevant = เรื่องใน test ที่ rating ≥ 4 (หลายเรื่องได้) ⟶ Precision@3 / Recall@3 / MAP@3 มีความหมายเต็ม ; ผู้ใช้ที่ test ไม่มี positive ถูกข้าม
- metric ทั้งหมดจาก Core เดิม ; ILD วัดใน `X_MODEL` (fused) ; Calibration เทียบกับ `train_r`
- ตัวเลขนี้เทียบกับ LAB 1-2 ได้เชิงโครงสร้าง แต่ **split ต่างกัน** (75/25 vs LOO) ต้องระบุเสมอ
- `best` เปลี่ยนเป็น RF ได้บรรทัดเดียว

จากคู่ (user, item) สู่ Top-K ด้วยโมเดล (Part 5–9)

X_MODEL = fused 2,688 มิติ SBERT + ViT + Wav2Vec2 + Video-ViT

train_r ของผู้ใช้ ไม่มี rating ใน test

p = make_profile(uid, X_MODEL, train_r, Rating-centered)

pair_feature ต่อ candidate [cos, |p−v|, p ⊙ v] = 5,377

Logistic.predict_proba ⟶ P(like)

sorted ⟶ Top-3 predictive_rank

relevant ใน test อยู่อันดับใด? P R F1 HR MRR MAP nDCG + Beyond

เทียบกับ LAB 1–2

- LAB 1–2: score = cosine(p, v) ใน fused space ;
  LAB 3: score = $\sigma(w \cdot [\cos, |p-v|, p \odot v] + b)$ ที่ **เรียนรู้ w** — โมเดลถ่วงน้ำหนักช่วงมิติของแต่ละสื่อเองได้ (learned modality weighting)
- ถ้า w ให้น้ำหนักเฉพาะ cos ⟶ กลับไปเท่ากับ LAB 1
- ตัวอย่างไม่กี่สิบแถว × 5,377 feature ⟶ ผลอาจ overfit ; ดู AUC ใน Part 10 หลายเงื่อนไขประกอบ

4

Experiment และ GUI

Part 10 – Part 11: Profile × Transformation × Model แล้วสาธิตพร้อมสื่อ

- `clone` โมเดล ; วัด AUC บน test เดิม
- GUI แสดง Train History, Recommendations (P(like)) และ image / audio / video ของอันดับ 1

Part 10: Transformation × Profile × Model ⟶ AUC

```python
from sklearn.base import clone
exp = []
for trans in ["L2", "Center+L2"]:
    X = transform_vectors(X_BASE, trans)
    for prof in PROFILE_METHODS:
        A, ya, ra, _ = make_pairs(train_r, X, train_r, prof)
        B, yb, rb, _ = make_pairs(test_r, X, train_r, prof)
        for name, base in CLF.items():
            m = clone(base); m.fit(A, ya); pp = m.predict_proba(B)[:, 1]
            exp.append({"Transformation": trans, "Profile": prof,
                    "Model": name,
                        "AUC": roc_auc_score(yb, pp) if
                                len(np.unique(yb)) == 2 else np.nan})
display(pd.DataFrame(exp).round(4))
```

อธิบายโค้ด

แยกผลของ representation ออกจาก algorithm

- split เดิม (`train_r`/`test_r`) ทุกเงื่อนไข ⟶ AUC ต่างเพราะ transformation / profile / model เท่านั้น
- `make_pairs(..., prof)` รับ profile เป็นพารามิเตอร์ ⟶ ไม่ต้องแก้ global `PROFILE_FOR_MODEL` (สะอาดกว่า Text LAB 3 ที่ global ค้าง)
- `clone(base)` = โมเดลใหม่ยังไม่ฝึก ⟶ ไม่ปนกับ `CLF` ที่ฝึกใน Part 8 ซึ่ง GUI ยังใช้
- 2 × 3 × 2 = 12 แถว วัด AUC pointwise ; ต้องการ ranking metric ⟶ ใส่ loop ของ Part 9 ในแต่ละรอบ
- Center+L2 บน fused space ลบทิศทางร่วมของ 4 สื่อพร้อมกัน ⟶ สังเกตว่า AUC ดีขึ้นไหม

Part 11: gui_pred — Predictive Multimodal Recommender (1/2)

```python
import gradio as gr

def gui_pred(uid, k):
    ranked, sc = predictive_rank(uid, best, X_MODEL, train_r,
            PROFILE_FOR_MODEL)
    top = ranked[:int(k)]
    hist = train_r[train_r["user_id"].astype(str) == str(uid)].merge(
        items[["item_id", "title", "genre"]], on="item_id",
                how="left")[["title", "genre", "rating"]]
    rec = []
    for j, iid in enumerate(top, 1):
        r = items.iloc[ITEM_INDEX[iid]]
        rec.append({"Rank": j, "Title": r["title"], "Genre": r["genre"],
                    "P(like)": round(float(sc[iid]), 4), "Synopsis":
                            r["synopsis"]})
    first = items.iloc[ITEM_INDEX[top[0]]] if top else None
```

อธิบายโค้ด

ความน่าจะเป็น + สื่อจริง

- ใช้ `predictive_rank` กับ `train_r` (ไม่ใช่ `ratings` เต็ม) ⟶ คำแนะนำอาจรวมเรื่องที่อยู่ใน test ของผู้ใช้ ⟶ เทียบกับ rating จริงใน `test_r` ได้
- คอลัมน์ **P(like)** ∈ [0, 1] จาก Logistic ที่ฝึกใน Part 8 — ความหมายต่างจาก Score (cosine) ของ LAB 1–2
- คืน path ของ **image / audio / video ของอันดับ 1** ให้ Gradio แสดง ⟶ ตีความเชิงคุณภาพว่าโมเดลเลือกเพราะสื่อใด
- input มีแค่ User และ Top-K เพราะ representation/profile/model ถูกตรึง (X_MODEL + Rating-centered + Logistic)
- แนวทางสอน: ดู Train History ⟶ ดู Top-K + สื่ออันดับ 1 ⟶ เปิด `test_r` ของผู้ใช้คนนั้นดูว่าเรื่องที่ซ่อนไว้ติดอันดับหรือไม่

Part 11: gui_pred — Predictive Multimodal Recommender (2/2)

```
    return (hist, pd.DataFrame(rec),
            first["image_path"] if first is not None else None,
            first["audio_path"] if first is not None else None,
            first["video_path"] if first is not None else None)

demo = gr.Interface(
    gui_pred,
    [gr.Dropdown(sorted(ratings["user_id"].unique()), label="User"),
            gr.Slider(1, 5, 3, 1, label="Top-K")],
    [gr.Dataframe(label="Train History"),
            gr.Dataframe(label="Recommendations"),
     gr.Image(label="Top-1 Image"), gr.Audio(label="Top-1 Audio"),
             gr.Video(label="Top-1 Video")],
    title="Predictive Multimodal Recommender")
demo.launch(debug=False)
```

อธิบายโค้ด

ความน่าจะเป็น + สื่อจริง

- ใช้ `predictive_rank` กับ `train_r` (ไม่ใช่ `ratings` เต็ม) ⟶ คำแนะนำอาจรวมเรื่องที่อยู่ใน test ของผู้ใช้ ⟶ เทียบกับ rating จริงใน `test_r` ได้
- คอลัมน์ **P(like)** ∈ [0, 1] จาก Logistic ที่ฝึกใน Part 8 — ความหมายต่างจาก Score (cosine) ของ LAB 1-2
- คืน path ของ **image / audio / video ของอันดับ 1** ให้ Gradio แสดง ⟶ ตีความเชิงคุณภาพว่าโมเดลเลือกเพราะสื่อใด
- input มีแค่ User และ Top-K เพราะ representation/profile/model ถูกตรึง (X_MODEL + Rating-centered + Logistic)
- แนวทางสอน: ดู Train History ⟶ ดู Top-K + สื่ออันดับ 1 ⟶ เปิด `test_r` ของผู้ใช้คนนั้นดูว่าเรื่องที่ซ่อนไว้ติดอันดับหรือไม่

สรุปแนวคิดของ Source Code Multimodal LAB 3

Take-away

- **Core เดิม + pipeline ใหม่:** ใช้ `make_profile`, `transform_vectors`, `early_fusion`, metric จาก LAB 1 แต่แทน `evaluate_vector_space` ด้วย loop ที่จัดอันดับด้วยโมเดล
- **Representation = Transformer-rich fusion** (SBERT + ViT + Wav2Vec2 + Video-ViT, 2,688 มิติ) ⟶ pair features 5,377 มิติ ; ช่วงมิติสะท้อนสื่อ ⟶ โมเดลเรียนรู้ modality weighting ได้
- **Split ระดับ interaction 75/25 ต่อผู้ใช้:** test มี label 0/1 และ rating จริง ⟶ ฝึก/วัด classifier กับ regressor และ relevant ต่อคนมีหลายเรื่อง
- **สองระดับของการวัด:** pointwise (Accuracy/F1/AUC, MAE/RMSE) และ ranking ต่อผู้ใช้ (P/R/F1/HR/MRR/MAP/nDCG + Beyond) — ดีระดับหนึ่งไม่รับประกันอีกระดับ
- **`pair_feature` รับ `profile_method` เป็นพารามิเตอร์** ⟶ Part 10 ทดลอง profile ได้โดยไม่พึ่ง global ; `clone` + split เดิม แยกผลของ representation จาก algorithm
- **ตีความอย่างระวัง:** ตัวอย่างไม่กี่สิบแถว × 5,377 feature ⟶ overfit ง่าย ; ใช้เพื่อเรียนรู้โครงสร้าง pipeline และ GUI แสดงสื่อจริงเพื่อตรวจเชิงคุณภาพ

Adapter ⟶ Vector ⟶ Fusion ⟶ Profile ⟶ Pair Features ⟶ Model ⟶ Ranking ⟶ Evaluation ⟶ GUI